

Assignment: Smart Day Planner Based on Weather Conditions

Overview

You are asked to design and implement a **Smart Day Planner** web service that proposes a daily activity plan based on **weather conditions**. Your application should use **Strategy** and **Observer** design patterns to dynamically adapt proposed activities when the weather changes.

The project simulates a real-world design problem involving multiple system components, API integration, persistence, background task scheduling, and containerized deployment.

Objectives

- Apply **Strategy** and **Observer** design patterns meaningfully.
- Use **FastAPI** for a modular, maintainable backend.
- Integrate **MongoDB** for persistence.
- Use **OpenWeatherMap API** for weather forecasts.
- Implement **scheduled weather checks** (via FastAPI background tasks or Celery).
- Containerize the app using **Docker** and **docker-compose**.
- Add **basic user preferences** to personalize activity planning.
- Build a **simple HTML/CSS frontend** to display current plans and weather (optional).

System Concept

Your application should:

1. Fetch weather forecast data for a given city using the **OpenWeatherMap API**.
2. Propose a **daily plan** based on current weather using **Strategy Pattern**.
3. **Automatically adapt** when weather changes (e.g., replace *Hiking* with *HouseWork* if it starts to rain).
4. Allow users to define **preferences** (e.g., “I prefer outdoor activities,” “Avoid sports on weekdays”).
5. Periodically **re-check weather data** in the background (either using FastAPI background tasks or Celery).
6. Store all plans, activities, weather data, and user preferences in **MongoDB**.
7. Expose a **FastAPI REST API** with endpoints for:
 - Retrieving the current plan
 - Forcing weather updates
 - Managing user preferences
 - Viewing stored plans
8. Include a **simple HTML/CSS interface** displaying:
 - Current weather
 - Current activity plan
 - Option to trigger plan refresh
9. Log all major events (data fetching, plan updates, errors, database operations).

Design Patterns Requirements

Strategy Pattern

- **Context:** DayPlanner
 - Uses a WeatherStrategy object to determine activities.
- **Strategy Interface:** WeatherStrategy
 - Defines a method `get_activities(user_preferences) -> list[Activity]`.
- **Concrete Strategies:**
 - SunnyWeatherStrategy
 - RainyWeatherStrategy
 - CloudyWeatherStrategy
 - SnowyWeatherStrategy
- Each strategy generates activities considering both **weather** and **user preferences**.

Observer Pattern

- **Subject (Observable):** WeatherStation
 - Fetches weather updates periodically (via background scheduler).
 - Notifies observers when the weather changes.
- **Observer:** DayPlanner
 - Responds to weather updates by regenerating the plan and updating MongoDB.

Background Scheduling

Option 1: Using FastAPI Background Tasks

- Use BackgroundTasks or a repeating task with asyncio/APScheduler to periodically fetch new weather data.
- When weather changes, trigger DayPlanner to update the plan.

Option 2: Using Celery (Advanced Option)

- Use **Celery** with Redis as a message broker to periodically run weather checks (e.g., every 30 minutes).
- Celery tasks communicate with the main FastAPI app through MongoDB or HTTP endpoints.

User Preferences

Each user can store preferences such as:

- Preferred activity types: ["outdoor", "productive"]
- Activities to avoid: ["sport", "date"]
- Working hours: {"start": 9, "end": 17}
- Weekend behavior: e.g., “Always relax on Sundays.”

Example MongoDB document:

```
{  
  "user_id": "12345",
```

```
"preferences": {  
  "preferred_types": ["outdoor", "learning"],  
  "avoid_types": ["sport"],  
  "working_hours": {"start": 9, "end": 17},  
  "weekend_mode": true  
}  
}
```

The planner should consider these preferences when choosing activities via the Strategy Pattern.

Data Model Example (MongoDB)

```
{  
  "date": "2025-11-07",  
  "location": "Berlin",  
  "weather": {  
    "condition": "Rainy",  
    "temperature": 12  
  },  
  "activities": [  
    {"name": "HouseWork", "type": "indoor", "priority": 3},  
    {"name": "Studying", "type": "productive", "priority": 2}  
  ],  
  "user_id": "12345"  
}
```

Logging

Implement a global logger for:

- Weather API calls

- Background task execution
- Activity plan updates
- Database operations

Example log output:

[INFO] 2025-11-07 08:00:00 Weather updated: Rainy

[INFO] 2025-11-07 08:00:01 Plan updated: Hiking -> HouseWork

[DEBUG] 2025-11-07 08:00:01 Inserted new plan into MongoDB

Suggested Project Structure

smart_day_planner/

```

|
|— app/
|   |— main.py          # FastAPI entry point
|   |— api/
|       |— routes.py    # API endpoints
|   |— core/
|       |— config.py    # Config & constants
|       |— logger.py    # Logger setup
|   |— weather/
|       |— weather_station.py # Observer (Subject)
|       |— weather_api.py  # OpenWeatherMap API integration
|   |— planner/
|       |— day_planner.py # Context + Observer
|       |— strategies/
|           |— base.py
|           |— sunny.py
|           |— rainy.py

```

```

| | | └─ cloudy.py
| | | └─ snowy.py
| | └─ activities/
| | | └─ base.py
| | | └─ housework.py
| | | └─ studying.py
| | | └─ sport.py
| | | └─ date.py
| | | └─ hiking.py
| └─ db/
| | └─ mongodb.py    # MongoDB connection
| | └─ models.py     # Schemas
| └─ tasks/
| | └─ scheduler.py  # FastAPI background or Celery tasks
| └─ frontend/
|   └─ templates/
|     └─ index.html  # Simple UI
|     └─ static/
|       └─ styles.css
|
└─ docker-compose.yml
└─ Dockerfile
└─ requirements.txt
└─ README.md

```

Docker Setup

Compose file must define:

- app (FastAPI app)
- mongo (MongoDB)

- Optional: redis (if using Celery)
- Optional: mongo-express (DB viewer)

Deliverables

1. **Source code** (GitHub repository)
2. **README.md** with:
 - Setup & run instructions
 - Example API usage
3. **Simple HTML/CSS frontend**
 - Displays current plan, weather, and “Refresh Plan” button

Bonus Ideas

- Add weather trend visualization in frontend (e.g., temperature chart).
- Store historical weather-activity correlations for recommendations.
- Add email/notification system when plans change significantly.
- Integrate a small authentication system (JWT).